Habit Mastery

Project One Report (Undergrad Submission)

Team: UG-P13

Repository: github.com/nwg02/MAD_project1

Report status: Draft based on implementation and checks reported on October 26, 2026

1. Project Overview

Habit Mastery is an offline-first Flutter application for defining habits, recording activity, and reviewing progress. Habit records and activity entries are stored locally in SQLite. User preferences are stored with SharedPreferences. The application also includes a local, explainable Next-Step Habit Suggestion feature that uses a deterministic rule rather than remote inference or a trained machine-learning model.

The project is intended to support routine tracking while keeping the user in control of target changes. Recommendations are presented as suggestions and are not saved automatically.

2. Goals and Scope

- Create, edit, activate, pause, and delete habits.
- Record and review activity entries for individual habits.
- Filter history and display progress summaries, including counts, totals, averages, and completion rates.
- Persist habit and activity data locally and retain user preferences between sessions.
- Provide a transparent rule-based target suggestion with an explanation and explicit user confirmation.
- Support offline use without cloud storage, remote authentication, or remote AI inference.

3. Implementation and Architecture

Interface: Flutter and Dart screens are implemented in lib/main.dart. Screen-level state manages loaded records, filters, settings, and temporary suggestion interactions.

Data layer: lib/data/habit_database.dart handles SQLite operations for habits and activity entries. The HabitEntry table references Habit through a foreign key, with cascading deletion intended to prevent orphaned activity records. Activity amounts are validated before insertion.

Suggestion logic: lib/data/habit_suggestion.dart contains the rule-based recommendation function separately from the interface. AI_FEATURE.md documents its inputs, rules, evaluation cases, baseline, and limitations.

Preferences: SharedPreferences stores settings such as compact history display and a reminder preference. The reminder preference does not itself enable operating-system notifications.

Documentation: README.md describes setup and features, ARCHITECTURE.md describes the structure and data flow, AI_Usage_Log.md discloses AI assistance, and verification responsibilities.

4. Next Step Habit Suggestion

The feature evaluates the current target and up to five recent activity amounts. If the target is invalid, history is empty, or no valid amounts remain, it retains the supplied target and explains the issue. Non-finite and negative activity amounts are excluded. Otherwise, the feature calculates the average of valid amounts. If the average meets or exceeds the target, it recommends a 10% increase, otherwise, it recommends a 10% decrease.

The recommendation appears with a reason and offers Apply and Not now. Apply opens the existing edit form with the proposed target prefilled and the change does not persist until the user confirms through the normal ‘Save’ action.

5. Testing and Verification

On October 26, 2026, the developer reported the following command results:

- flutter analyze: No issues found (ran in 1.7 seconds)
- flutter test: All 17 tests passed (reported runtime about 4 seconds)
- git status: On branch main, up to date with origin/main, and working tree clean

The automated suite includes database behavior, habit CRUD, activity entry operations, filtering, widget behavior, and suggestion logic. The reported test run confirms the current automated suite passed but it does not verify by itself every rubric requirement or all manual workflows.

6. Suggestion Evaluation Plan and Limitation

The documented evaluation uses five deterministic scenarios: normal recent history, no history, invalid target, boundary/invalid activity values, and a counterexample with repeated zero-value entries. These scenarios assess rule consistency and input handling, not whether recommendations improve long-term habit completion. The five cases are software test scenarios, not a representative user-study sample. The heuristic cannot explain why activity was missed, understand user circumstances, or determine whether a target is appropriate. An average may be influenced by unusual entries. In the counterexample, repeated zero amounts cause the rule to recommend reducing the target, which may not address the underlying reason for inactivity.

7. Version Control and Documentation

The Git log provided on October 26, 2026, includes commits for database validation, suggestion logic and integration, automated suggestion tests, and project documentation. The repository was reported as synchronized with origin/main with a clean working tree. The supplied log output was truncated/repeated by the terminal pager, so this report does not assert a complete contribution count or independently verify all branch, pull-request, or Kanban requirements. Files present in the project root at the time of reporting included README.md, ARCHITECTURE.md, AI_FEATURE.md, and AI_Usage_Log.md. The approved proposal was found one directory above the project as Flutter_Build_Studio_Proposal_UG-P13.pdf.

8. Remaining Verification and Submission Items

This report is a draft and should be updated before final submission. The following items have not been independently evidenced in the information available for this report: • Manual testing on two devices or emulators, including restart persistence and offline operation. • Peer usability observation, resulting design change, and retest evidence, if required by the instructor. • Dated ClickUp board snapshots and task-to-code-to-test contribution evidence. • Final release APK built and tested from the current source. • Presentation slides, questions-only Word document, required signed acknowledgment, final individual ZIP, and video link/evidence. • Confirmation that all required files are included in the repository and each individual submission package. Do not represent these items as completed until the corresponding evidence exists. The report should be revised to include actual results, screenshots, and links where the assignment requires them.

9. Conclusion

Habit Mastery currently has an offline-first Flutter implementation with local SQLite persistence, preference storage, progress summaries, and an explainable rule-based habit target suggestion. The reported static analysis and automated tests pass. Final readiness depends on completing and documenting the remaining manual verification and course submission evidence